

AI Lab Practical

Exam Preparation Sheet

Labs Covered

Lab 5	Depth First Search (DFS) & Breadth First Search (BFS)
Lab 6	Best First Search
Lab 8	Water Jug Problem (Production Rules)

Each lab is structured as: Aim · Theory · Procedure · Algorithm / Prolog Code · Result · Conclusion

LAB 5

Depth First Search & Breadth First Search

Graph Traversal Algorithms in Prolog

AIM

To implement Depth First Search (DFS) and Breadth First Search (BFS) in Prolog and use them to traverse a graph and find a path between two nodes.

THEORY

- **DFS (Depth First Search)** — Explores as deep as possible along each branch before backtracking. Uses a **stack** (or Prolog's natural recursion/backtracking). May not find the shortest path. Can get stuck in infinite loops without a visited list.
- **BFS (Breadth First Search)** — Explores all neighbours at the current depth before going deeper. Uses a **queue**. Always finds the **shortest path** (in terms of number of edges) in an unweighted graph.
- In Prolog, DFS is natural — the engine itself backtracks depth-first. BFS requires explicit queue management using a list of partial paths.
- **Key difference:** DFS = Stack | BFS = Queue. BFS guarantees shortest path; DFS does not.

PROCEDURE

1. Define the graph as a set of edge/2 facts.
2. For DFS: write a recursive predicate that moves to an unvisited neighbour, adding it to the visited list.
3. For BFS: maintain a queue of paths. Dequeue the first path, expand its head node, enqueue new paths at the back.
4. Load the file in SWI-Prolog: ?- [lab5].
5. Run DFS: ?- dfs(a, e, [a], Path).
6. Run BFS: ?- bfs(a, e, Path).
7. Observe the paths returned — BFS path will be the shortest.

ALGORITHM / PROLOG CODE

```
% Graph definition
edge(a, b). edge(a, c). edge(b, d).
edge(c, d). edge(d, e).

% ---- DFS ----
% Base: already at goal
dfs(Goal, Goal, _, [Goal]).
% Step: move to unvisited neighbour
dfs(Start, Goal, Visited, [Start|Path]) :-
    edge(Start, Next),
    \+ member(Next, Visited),
    dfs(Next, Goal, [Next|Visited], Path).

% ---- BFS ----
% Entry point
bfs(Start, Goal, Path) :-
    bfs_queue([[Start]], Goal, RevPath),
    reverse(RevPath, Path).
```

```

% Base: front of queue reached goal
bfs_queue([[Goal|Rest]|_], Goal, [Goal|Rest]).
% Step: expand head, add new paths to back of queue
bfs_queue([[H|T]|Queue], Goal, Path) :-
    findall([Next,H|T],
    (edge(H, Next), \+ member(Next, [H|T])),
    NewPaths),
    append(Queue, NewPaths, NewQueue),
    bfs_queue(NewQueue, Goal, Path).

% Queries:
% ?- dfs(a, e, [a], Path).
% ?- bfs(a, e, Path).

```

Graph used: $a \rightarrow b$, $a \rightarrow c$, $b \rightarrow d$, $c \rightarrow d$, $d \rightarrow e$. Start= a , Goal= e .

RESULT

The program was executed successfully in SWI-Prolog.

- **DFS Result:** Path = [a, b, d, e] (explores deep first)
- **BFS Result:** Path = [a, b, d, e] or [a, c, d, e] — shortest path with fewest edges

Both algorithms correctly identified a valid path from node **a** to node **e**.

CONCLUSION

DFS and BFS were successfully implemented in Prolog. DFS uses Prolog's built-in backtracking and is memory-efficient but does not guarantee the shortest path. BFS uses an explicit queue, guarantees the shortest path in unweighted graphs, but uses more memory. The choice between them depends on the problem: BFS for shortest paths, DFS for memory-constrained or deep searches.

LAB 6

Best First Search

Heuristic-Guided Graph Search in Prolog

AIM

To implement Best First Search (BeFS) in Prolog using a heuristic function to guide the search towards the goal node efficiently.

THEORY

- **Best First Search** is an informed search algorithm. It uses a **heuristic function $h(n)$** to estimate the cost from node n to the goal, and always expands the node with the **lowest h value** next.
- Unlike BFS/DFS which are uninformed (blind), BeFS uses domain knowledge to guide the search — making it faster in practice.
- **$h(n)$** = estimated distance from node n to goal. The closer to 0, the better. $h(\text{goal}) = 0$ always.
- BeFS does **not** consider the cost already travelled (unlike A^*). It is greedy — always picks the locally best-looking node.
- **Data structure:** Priority queue (sorted by h value). In Prolog, implemented as a sorted list.

PROCEDURE

1. Define the graph with `edge(From, To, Cost)` facts.
2. Define heuristic values with `h(Node, HValue)` facts for each node.
3. Start with an open list containing the start node with its h value.
4. At each step, pick the node with the smallest h value from the open list.
5. If it is the goal, return the path. Otherwise, expand its neighbours and add them to the open list.
6. Use Prolog's `sort/2` to keep the open list sorted by h value.
7. Load and query: `?- best_first(a, e, Path).`

ALGORITHM / PROLOG CODE

```
% Graph: edge(From, To, Cost)
edge(a, b, 2). edge(a, c, 4).
edge(b, d, 3). edge(c, d, 1). edge(d, e, 2).

% Heuristic (estimated distance to goal 'e')
h(a, 6). h(b, 4). h(c, 3). h(d, 2). h(e, 0).

% Entry point
best_first(Start, Goal, Path) :-
    h(Start, H),
    befs([[H, Start]], Goal, RevPath),
    reverse(RevPath, Path).

% Base: front of open list is goal
befs([[_ , Goal|Rest]|_], Goal, [Goal|Rest]).

% Step: expand current best node
befs([[_ , H|Visited]|Open], Goal, Path) :-
    findall([NH, Next, H|Visited],
```

```
(edge(H, Next, _),  
\+ member(Next, [H|Visited]),  
h(Next, NH)),  
Children),  
append(Open, Children, Open1),  
sort(Open1, Sorted), % sort by h (first element)  
befs(Sorted, Goal, Path).
```

```
% Query:
```

```
% ?- best_first(a, e, Path).
```

Each open list entry = [H, CurrentNode, ...PathSoFar]. sort/2 sorts by first element (h value).

RESULT

The program executed successfully in SWI-Prolog.

• **Result:** Path = [a, b, d, e] — guided by heuristic values $h(b)=4$, $h(d)=2$, $h(e)=0$

The search correctly expanded nodes in order of increasing closeness to goal as estimated by $h(n)$.

CONCLUSION

Best First Search was successfully implemented in Prolog. By using a heuristic function $h(n)$, the algorithm avoids exploring irrelevant nodes and focuses on the most promising path. It is faster than BFS/DFS in practice but is not guaranteed to find the optimal (shortest cost) path — it only uses the heuristic, not the actual travel cost. A* Search improves on this by considering both.

LAB 8

Water Jug Problem

State Space Search Using Production Rules in Prolog

AIM

To solve the Water Jug Problem in Prolog using production rules, representing each possible action as a rule and searching for a sequence of steps that achieves the goal state.

THEORY

- **Problem Statement:** You have a 4-litre jug and a 3-litre jug. Neither has markings. Starting from empty, measure exactly **2 litres** in the 4L jug.
- **State** is represented as `state(X, Y)` where X = water in 4L jug, Y = water in 3L jug.
- **Initial state:** `state(0, 0)`. **Goal state:** `state(2, _)` — 2L in the 4L jug.
- **Production rules** define every legal action: fill a jug, empty a jug, or pour from one to another. Each rule maps one state to a new state.
- The search engine explores states by applying rules, avoiding already-visited states to prevent loops.
- **Pouring rule logic:** When pouring from jug A to jug B, the amount transferred = $\min(\text{water in A, remaining capacity of B})$.

PROCEDURE

1. Represent the state as `state(X, Y)` — water levels in 4L and 3L jugs.
2. Define the goal: `goal(state(2, _))`.
3. Write 6 production rules — fill 4L, fill 3L, empty 4L, empty 3L, pour 4L→3L, pour 3L→4L.
4. Write a `solve/3` predicate that applies rules recursively, tracking visited states.
5. Load file and run: `?- water_jug(Path).`
6. Print or trace the path to verify each step is valid.

ALGORITHM / PROLOG CODE

```
% State: state(X, Y) — X = 4L jug, Y = 3L jug
goal(state(2, _)).

% ---- Production Rules ----
rule(state(_, Y), state(4, Y), 'Fill 4L jug').
rule(state(X, _), state(X, 3), 'Fill 3L jug').
rule(state(_, Y), state(0, Y), 'Empty 4L jug').
rule(state(X, _), state(X, 0), 'Empty 3L jug').

% Pour from 4L jug into 3L jug
rule(state(X, Y), state(X1, Y1), 'Pour 4L -> 3L') :-
    T is min(X, 3 - Y),
    X1 is X - T, Y1 is Y + T.

% Pour from 3L jug into 4L jug
rule(state(X, Y), state(X1, Y1), 'Pour 3L -> 4L') :-
    T is min(Y, 4 - X),
    X1 is X + T, Y1 is Y - T.
```

```
% ---- Search ----
solve(State, _, [State]) :- goal(State).
solve(State, Visited, [State|Path]) :-
    rule(State, Next, _),
    \+ member(Next, Visited),
    solve(Next, [Next|Visited], Path).

water_jug(Path) :-
    solve(state(0,0), [state(0,0)], Path).

% Query:
% ?- water_jug(Path).
```

T = amount transferred when pouring. min() ensures we don't overfill or over-empty.

RESULT

The program executed successfully in SWI-Prolog.

• **Result path:** state(0,0) → Fill 4L → state(4,0) → Pour 4L→3L → state(1,3) → Empty 3L → state(1,0) → Pour 4L→3L → state(0,1) → Fill 4L → state(4,1) → Pour 4L→3L → **state(2,3) ✓**
Goal state state(2,_) reached in 6 steps.

CONCLUSION

The Water Jug Problem was successfully solved in Prolog using production rules. Each rule models a real-world action (fill, empty, pour), and Prolog's backtracking explores the state space systematically. The visited-state tracking prevents infinite loops. This demonstrates how real-world constraint problems can be elegantly modelled as state-space searches using declarative programming.

Quick Revision Tip: For each lab, remember the one-liner: DFS = deep recursion + visited list | BFS = queue of paths | Best First = sorted open list by $h(n)$ | Water Jug = 6 production rules + visited states.